1-1 Loop Engineering：让AI Agent自己转起来的工程方法论

> [!TIP] 阅读指南 前面几章，我们学会了怎么给 Agent 写 prompt、怎么配置 Skill、怎么搭建 MCP 连接器。但一个更根本的问题来了：**如果每次都要你自己发指令，Agent 再强又有什么用？**Loop Engineering 是 2026 年夏天突然火起来的一个新概念。它的核心主张——把你从“操作员”变成“监督员”，让 Agent 自己按循环自动运转。这一节我们从故事讲起，逐步拆解它的六个积木块、三个核心文件、八个 CLI 工具，以及它和 Goal Engineering、Shell Engineering 的关系。

1.1 先讲一个故事

回想一下你平时怎么用 AI 的。

“帮我把这些截图转成 WebP。”“帮我把这个 issue 分类一下。”“看看 npm 依赖有没有更新。”

你有没有发现，你在做一件事：**你发现问题，然后告诉 AI 去解决。**

你发现截图要转格式，你告诉 AI。你发现 issue 没人处理，你告诉 AI。你发现依赖该更新了，你告诉 AI。你像是一个“问题搬运工”——把现实世界里的问题，一条条搬进对话框里。

这个模式有个隐藏的成本：**你得一直在场。**

你不在，AI 不会自己发现“哦，今天有新 issue 了”。你没想到，AI 不会主动说“哎，依赖该升了”。你跟 AI 的关系，本质上是你在驱动它——你发现问题，它解决问题。

那能不能反过来？

不是“你发现问题，告诉 AI 去解决”，而是让 AI **自己去发现问题、决定优先级、动手解决、最后告诉你结果。**

你给它一个角色，而不是一个任务：

> 你的工作是维护这个项目的健康度。
>
> 每天早上 9 点，你自己去看 GitHub 上有没有新 issue，判断要不要处理。
>
> 每 4 小时，检查依赖有没有更新，小版本补丁自己升，大版本升之前问我。
>
> 每次有人提 PR，你自己去跑测试，过不了就打回去，过了就告诉我一声。
>
> 你不确定的事，记在日志里，我每天看一次。

看到了吗？区别在于——**你没有告诉它“今天要干什么”，而是告诉它“你的工作是什么”。** 它自己决定今天该干什么。你不需要发现问题了再告诉它，因为发现问题本来就是它的工作。

Loop Engineering 就是给 AI 定义“岗位职责”，而不是给它下“任务指令”。

1.2 谁点燃了这把火

2026 年 6 月，两个背景完全不同的人，几乎同时说了同一件事。

Peter Steinberger，奥地利开发者，PSPDFKit 创始人（2021 年出售），2026 年初加入 OpenAI。他在 X 上发了一条帖子：

“你不应该再亲自给 coding agent 写 prompt 了。你应该设计 loop，让 loop 去 prompt agent。”

这条帖子迅速引爆了开发者社区。**来源：Addy Osmani 博客引用。**

Boris Cherny，Anthropic 公司 Claude Code 的负责人，在同一周接受采访时说了类似的话：

“我已经不亲自 prompt Claude 了。我有一堆 loop 在跑，它们自己 prompt Claude，自己决定要干什么。我的工作是写 loop。”

\[来源同上]

两个人，一个在 OpenAI，一个在 Anthropic，做的产品不同，却得出了同一个结论——**最会用 Agent 的人，已经不跟 Agent 直接对话了。** 他们写“脚本”，Agent 按脚本自己演。

这个想法很快有了工程实现。就在同一时期，**Cobus Greyling** 在 GitHub 上创建了 `loop-engineering` 仓库，把这套理念变成了可用的 CLI 工具。而 **Addy Osmani**（Google Chrome 工程总监，后任 Google Cloud AI 总监）也写了博客详细介绍了这个思路，让 Loop Engineering 进入了更多开发者的视野。

但听完这些来龙去脉，你可能更关心一个实际的问题：它到底能解决什么？

1.3 它在解决什么问题

现在的用法有什么问题

现在大多数人用 AI Agent 还是这样的：

代码块

```
你：“帮我看一下这个 GitHub issue，分类一下”
Agent：看了，是个 bug 报告，优先级 P1

你：“检查一下 npm 依赖有没有更新”
Agent：检查了，lodash 有一个安全更新

你：“PR #42 的测试跑过了吗？帮我 review 一下”

...
```